



UNIVERSIDAD DE SEVILLA
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Grado en Ingeniería Informática - Tecnologías Informáticas
Curso 2026/2027

PAI-1

Grupo 33

Realizado por Daniel Enrique Molina Sanglade, Carlos Cobos Gordillo
y Ángela Castilla López

Índice

1. Resumen ejecutivo.....	3
2. Diseño del protocolo.....	4
3. Análisis de seguridad y evidencias.....	5
3.1 Inyección de ataque MitM.....	5
3.2 Prueba de ataque Replay.....	5
3.3 Verificación de tiempo constante.....	6
4. Manual de Despliegue y Ejecución.....	6
5. Matriz de Trazabilidad de Requisitos.....	7
6. Anexo.....	8
6.1 Capturas de pantalla.....	8
6.2 Uso de IA.....	10

1. Resumen ejecutivo

Una entidad financiera nos ha encomendado el desarrollo e implementación de una arquitectura cliente-servidor para ofrecer un servicio seguro de transferencias bancarias a sus clientes, garantizando la integridad, autenticidad y confidencialidad de las transacciones. Se pide expresamente que la seguridad de las comunicaciones se implemente en la capa de aplicación, sin tocar la capa de transporte.

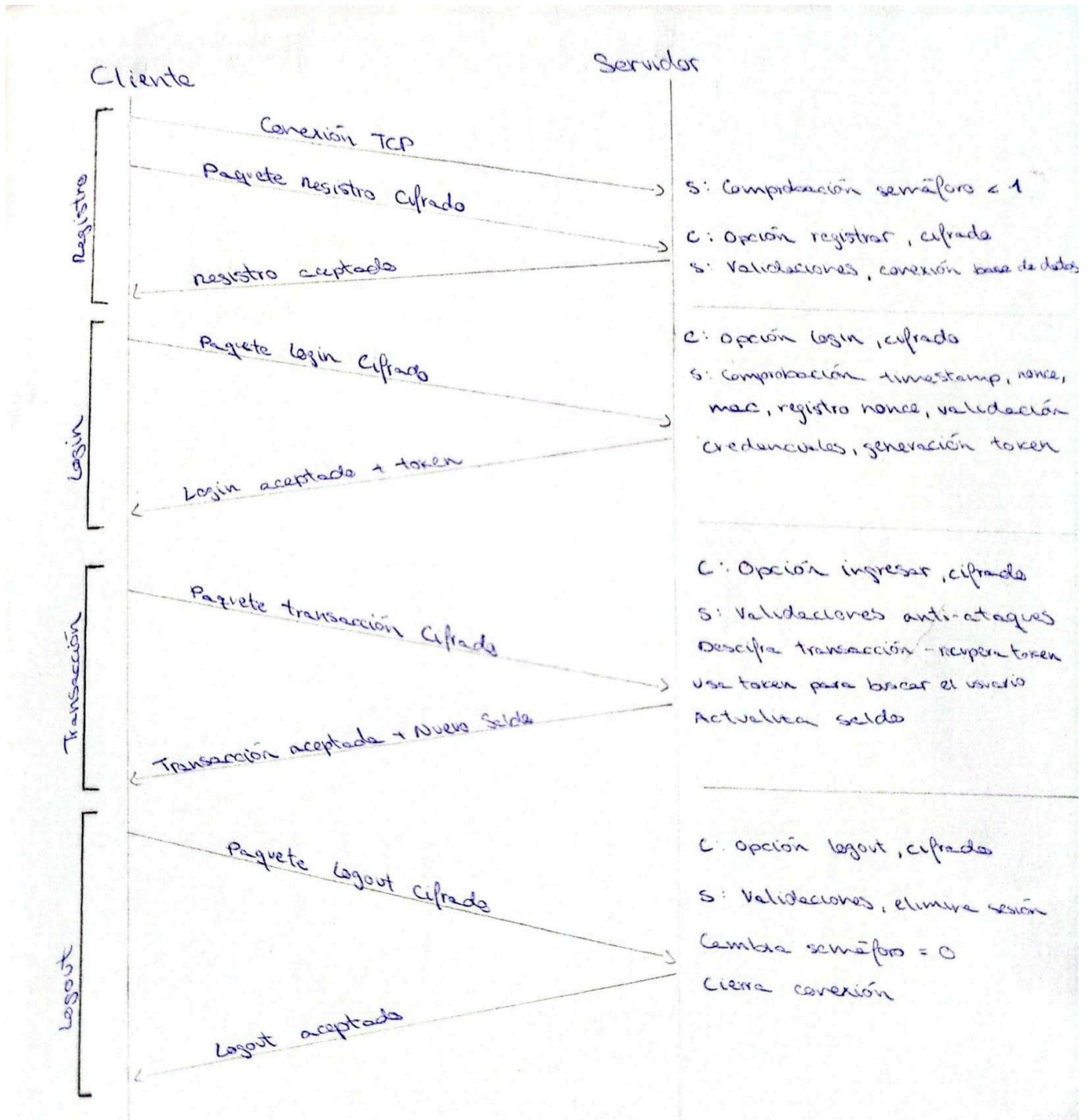
Para realizar este encargo, se ha diseñado una plataforma cliente-servidor basada en TCP Sockets junto a una base de datos SQL. Para cubrir todos los campos hemos realizado los siguientes puntos:

- Autenticidad e Integridad: Generación de firmas HMAC-SHA256 sobre el paquete de datos para prevenir ataques Man in the Middle.
- Confidencialidad: Cifrado simétrico del paquete mediante Fernet (AES-128-CBC) con claves derivadas dinámicamente mediante Argon2id y salt aleatorio.
- Uso combinado de marcas de timestamps de 60 segundos y nonces almacenados para impedir los ataques Replay.
- Implementación de comparaciones en tiempo constante evitando los ataques de canal lateral.

Para comprobar la efectividad de las medidas implementadas se han realizado pruebas de distintos tipos de ataque, verificando la robustez de la estructura creada.

2. Diseño del protocolo

Diagrama de secuencia de flujo de paquetes cliente-servidor en la capa de aplicación.



3. Análisis de seguridad y evidencias

Para demostrar el correcto funcionamiento de las medidas de seguridad implementadas en el sistema hemos llevado a cabo pruebas de distintos tipos de ataque.

3.1 Inyección de ataque MitM

Para asegurar la autenticidad del mensaje, se utiliza HMAC-SHA256 sobre el paquete. De esta forma, si ocurre un ataque de tipo Man-in-the-Middle que intenta alterar cualquier dato durante la transmisión, la firma HMAC deja de ser válida. Al llegar al servidor, la firma no coincide y el paquete es rechazado en el acto, impidiendo que la transacción manipulada se procese.

```
PS C:\Users\Pc\Desktop\Proyecto1-SSII> & C:\Users\Pc\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Pc/Desktop/Proyecto1-SSII/client/hacker.py
--- SIMULACIÓN DE ROBO DE TRÁFICO ---
Paquete capturado en texto plano por el atacante:
{"nonce": "fe2de8bb6215a3e2d728319ea5c3bd02", "timestamp": 1791185503, "mensaje_cifrado": "gAAAAABqw..."}

[>] Ejecutando: Paso original (Víctima enviando petición)
[+] ÉXITO DEFENSIVO: El servidor bloqueó el ataque -> Sesión no válida o expirada
[>] Ejecutando: Replay Attack (Reenviando el mismo paquete interceptado)
[+] ÉXITO DEFENSIVO: El servidor detectó el ataque y cortó la conexión silenciosamente.

[>] Ejecutando: MitM Attack (Enviando paquete con ciphertext alterado)
[+] ÉXITO DEFENSIVO: El servidor detectó el ataque y cortó la conexión silenciosamente.

--- PRUEBA 3: ATAQUE DE CANAL LATERAL DE TIEMPO ---
Media tiempo usuario EXISTENTE: 2.08571 seg
Media tiempo usuario FALSO: 2.08261 seg
Diferencia detectada: 0.00310 seg
[+] SEGURO: El servidor responde en tiempo constante. No se filtra información.
```

Simulación realizada con hacker.py

3.2 Prueba de ataque Replay

Para proteger la integridad del mensaje frente a ataques Replay, el servidor trabaja con un timestamp de 60 segundos, junto con la comprobación de que el nonce no esté repetido. Para esto, se ha creado un set que almacena los nonces utilizados. Si el servidor detecta un nonce repetido o un paquete caducado corta la conexión de inmediato.

```

PS C:\Users\Pc\Desktop\Proyecto1-SSII> & C:\Users\Pc\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Pc/Desktop/Proyecto1-SSII/client/hacker.py
--- SIMULACIÓN DE ROBO DE TRÁFICO ---
Paquete capturado en texto plano por el atacante:
{"nonce": "fe2de8bb6215a3e2d728319ea5c3bd02", "timestamp": 1791185503, "mensaje_cifrado": "gAAAAABqw..."}

[>] Ejecutando: Paso original (Víctima enviando petición)
[+] ÉXITO DEFENSIVO: El servidor bloqueó el ataque -> Sesión no válida o expirada
[>] Ejecutando: Replay Attack (Reenviando el mismo paquete interceptado)
[+] ÉXITO DEFENSIVO: El servidor detectó el ataque y cortó la conexión silenciosamente.

[>] Ejecutando: MitM Attack (Enviando paquete con ciphertext alterado)
[+] ÉXITO DEFENSIVO: El servidor detectó el ataque y cortó la conexión silenciosamente.

--- PRUEBA 3: ATAQUE DE CANAL LATERAL DE TIEMPO ---
Media tiempo usuario EXISTENTE: 2.08571 seg
Media tiempo usuario FALSO: 2.08261 seg
Diferencia detectada: 0.00310 seg
[+] SEGURO: El servidor responde en tiempo constante. No se filtra información.

```

Simulación realizada con hacker.py

3.3 Verificación de tiempo constante

Para evitar los ataques de canal lateral se ha implementado un comparador en tiempo constante con el que comprobamos que el mac recibido sea el esperado. A diferencia del operador convencional “==”, que compara byte a byte y cesa en cuanto detecta un carácter que no coincide, esta herramienta recorre todos los bytes independientemente de cuando detecte el fallo, asegurando así que todas las comparaciones duren el mismo tiempo. Esto impide que un atacante pueda deducir la diferencia de bytes analizando las diferencias de tiempo de respuesta.

```

if not secrets.compare_digest(paquete_recibido["mac"], mac_esperado):
    raise ValueError("Error de Integridad: Firma MAC inválida o paquete alterado.")

```

4. Manual de Despliegue y Ejecución

Para ejecutar el sistema hay que realizar los siguientes pasos:

En primera instancia, hay una serie de librerías que deben estar instaladas en los equipos de comunicación (algunas como cryptography, incluso en su misma versión), estas son: cryptography, mysql-connector-python y numpy. (El comando para su instalación será "pip install xxx", requiriendo por supuesto de alguna versión moderna de python >=3.15)

Tras haber descargado estas librerías, hay que encender la base de datos, para ello usaremos XAMPP para encender un servidor SQL alojado en el puerto 3306 (por defecto), con usuario "root", sin contraseña y cuya IP es localhost (127.0.0.1). Dentro de este servidor usaremos el script sql "CreaciónDB" para la creación simple de los usuarios "admin" y "cliente".

Como tercer paso, es necesario centrarnos en todos los scripts del proyecto y en concreto en "cliente.py", "servidor_enc.py" y "hacker.py". Usaremos "servidor_enc.py" para iniciar el servidor en el dispositivo que haya iniciado la base de datos (basta con ejecutar el script), mientras que desde otro dispositivo usaremos "cliente.py" (o hacker.py si queremos hacer distintas pruebas de seguridad) poniendo la IP de nuestro servidor en el apartado del script indicado para ello (puede saberse usando el comando "ipconfig" si estás en windows), es importante que ambos dispositivos sean visibles entre ellos, por lo que deben estar conectados a la misma red.

Por último, hay que asegurarse que en los archivos "seguridad.py" de las carpetas "client" y "server", la contraseña secreta es la misma. Si todas las comprobaciones son correctas, podemos ejecutar el script "cliente.py" que encontrará al servidor y podremos ejecutar nuestros comandos.

5. Matriz de Trazabilidad de Requisitos

	Fichero	Línea
Requisito funcional 1	a) servidor_enc.py	66-85
	b) CreacionDB.sql	2-19
	c) CreacionDB.sql	9
	d) servidor_enc.py	88-131
Requisito funcional 2	seguridad.py	68, 69
Requisito de Seguridad 1	a) seguridad.py	17-34
	b) servidor_enc.py	14, 15
Requisito de Seguridad 2	a) seguridad.py	18, 26
	b) seguridad.py	17, 18

Requisito de Seguridad 3	a) seguridad.py	22, 23
	b) seguridad.py	11
Requisito de Seguridad 4	seguridad.py	68, 69

6. Anexo

6.1 Capturas de pantalla

```

--- INICIANDO PRUEBA DE ROBUSTEZ ARGON2ID ---
Tiempo total para 10 intentos: 0.5689 segundos
Tiempo medio por derivación: 0.0569 segundos
Costo de memoria RAM por intento: 64.0 MB
Velocidad máxima del atacante: 18 combinaciones/segundo (por núcleo)
PS C:\Users\Pc\Desktop\Proyecto1-SSII>

```

Prueba de robustez realizada con el fichero prueba_derivación.py

```

PS C:\Users\Pc\Desktop\Proyecto1-SSII> & C:\Users\Pc\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Pc/Desktop/Proyecto1-SSII/client/cliente.py
[+] Cliente conectado con éxito al servidor banco

[--- INICIAR SESIÓN ---]
Usuario: admin
Contraseña: 1234

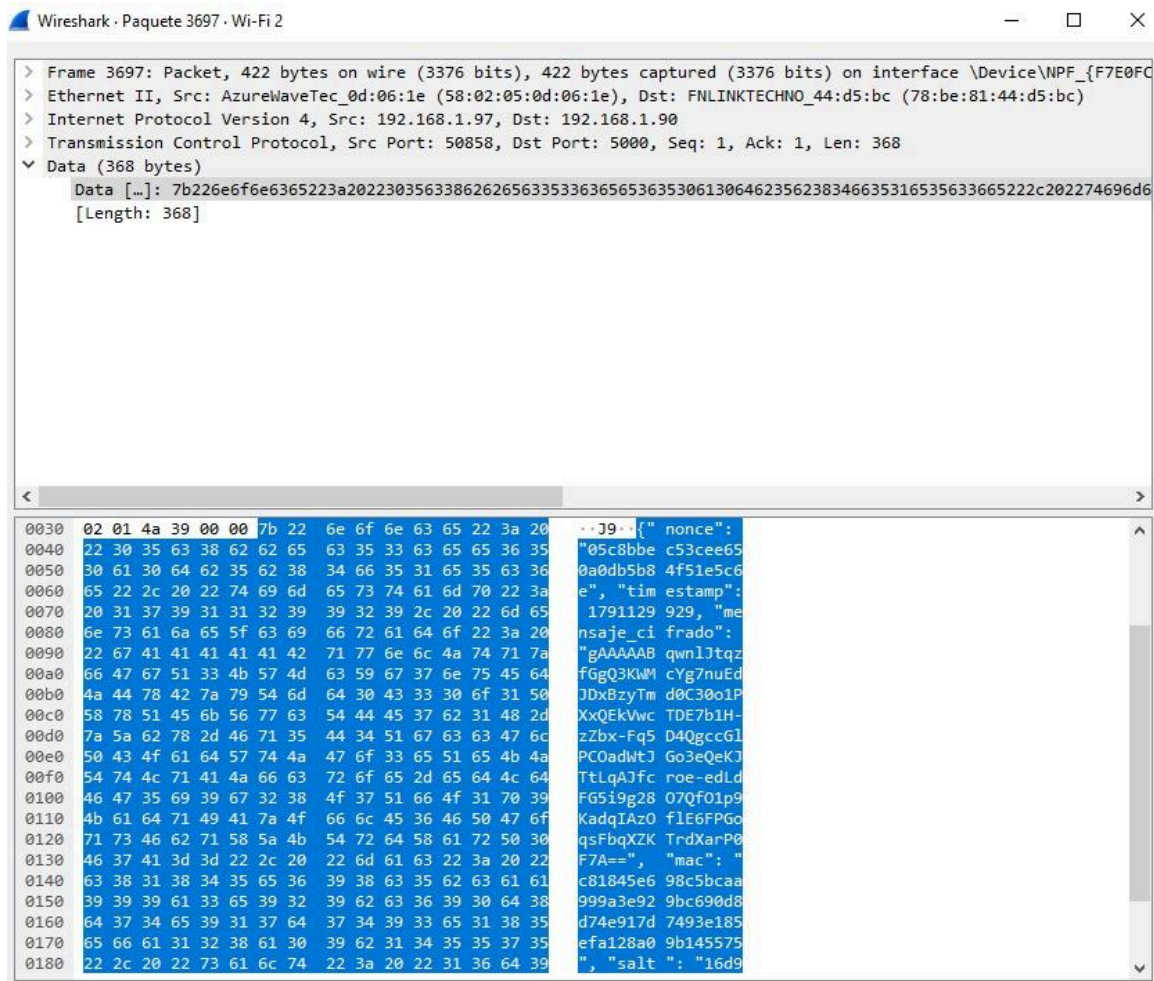
[+] Login exitoso. Saldo actual: 100.00€

(admin) > ingresar 100
[+] Has ingresado 100.00€. Nuevo saldo: 200.00€

(admin) >

```

Login e ingreso exitoso



Traza en Wireshark: mensaje encriptado

```
[+] Cliente conectado con éxito al servidor banco

[--- MENÚ PRINCIPAL ---]
1. Iniciar sesión
2. Crear nueva cuenta
3. Salir
Elige una opción (1/2/3): 2
Nuevo usuario: paco
Nueva contraseña: contraseña

[+] Cuenta 'paco' creada con éxito. Saldo inicial: 0.00€
```

Creación exitosa de nuevo usuario

```
[--- MENÚ PRINCIPAL ---]
1. Iniciar sesión
2. Crear nueva cuenta
3. Salir
Elige una opción (1/2/3): 2
Nuevo usuario: paco
Nueva contraseña: contraseña

[-] El nombre de usuario 'paco' ya está en uso.
```

Comprobación de conflicto de usuarios iguales

6.2 Uso de IA

Se ha utilizado la herramienta Gemini para la generación de la mayoría del código. Aunque el código se haya generado con IA, se han realizado revisiones exhaustivas, modificaciones y reestructuraciones a mano, conexiones entre distintos ficheros y una comprensión profunda del mismo.

Enlace a la conversación: <https://share.gemini.google/cdCUh9z5MVG3> ;
<https://gemini.google.com/share/d/1iuI2LIgncOVtM8nxxTXi26mIZXZbMkvO?usp=s>
haring